

Payment System Design

Core functions and objectives of the payment system:

- Design a payment processing system similar to Stripe that can handle various payment methods for online transactions
- The system needs to be secure, reliable, highly available, and able to process transactions globally
- Support merchant integration, allowing third-party applications to call our payment services via API

Key Functional Requirements:

- **Payment Processing:** Accept and process credit cards, debit cards, digital wallets, and other payment methods
- **Third-party Developers Integration:** Provide APIs for Third-party Developers to easily integrate payment functions
- **Payment Verification:** Verify payment information and fraud prevention systems
- **Payment Status Tracking:** Real-time tracking of payment status (pending, successful, failed)
- **Refund Processing:** Support full or partial refunds
- **Reports and Analytics:** Provide transaction history and financial reports for merchants
- **Multi-currency Support:** Process transactions in different currencies
- **Management Console:** Interface for merchants to manage transactions and configurations

Non-functional Requirements:

Scalability

- System needs to handle peak traffic (such as during shopping festivals)
- Estimated daily transaction volume: Assume processing 10 million transactions per day
- Average QPS: Approximately 115 QPS, with peak potentially 10 times the average
- System needs to scale horizontally to accommodate growth

Availability

- Requires 99.99% uptime (less than 52 minutes of downtime per year)
- System must be able to tolerate partial failures
- High availability is essential because payment is a critical business function

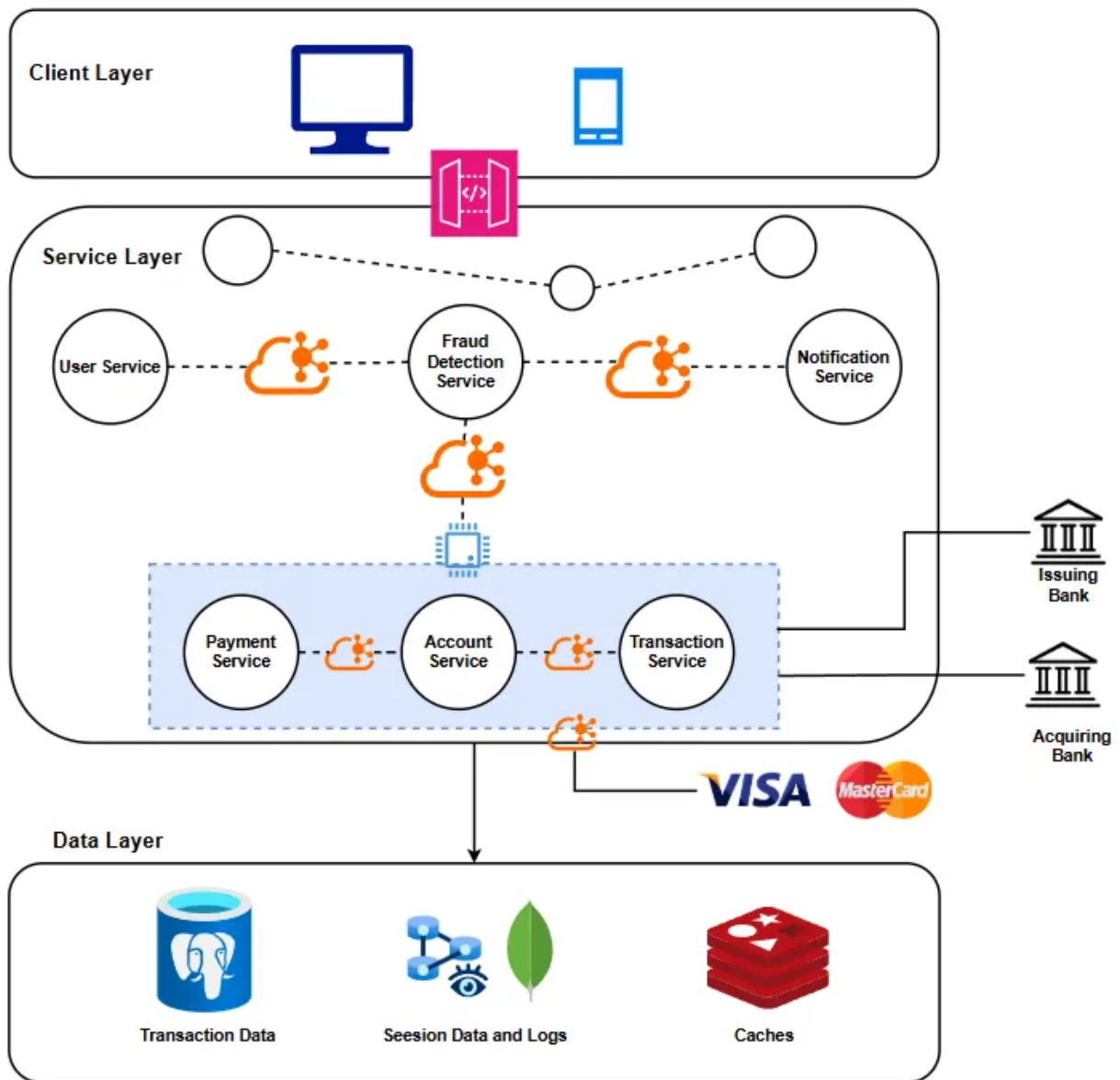
Data Modeling

- **Structured Data:** User information, transaction records, payment methods, etc.
- **Semi-structured Data:** API request/response logs, etc.
- **Unstructured Data:** Verification documents uploaded by users, etc.

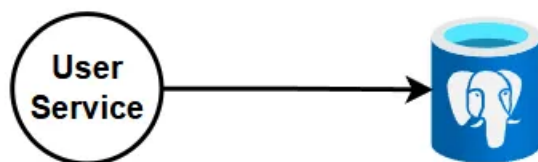
Rough Estimation

- Storage Requirements:
 - Assume each transaction record is about 1KB
 - 10 million transactions per day = 10GB/day
 - Approximately 3.65TB of raw data per year
 - Considering backups and redundancy, actual need is about 10–15TB/year
- Bandwidth Requirements:
 - Average transaction size approximately 2KB (request+response)
 - $115 \text{ QPS} * 2\text{KB} = 230\text{KB/s}$ average
 - Peak approximately 2.3MB/s
- Server Estimation:
 - Payment service: Each server handles about 1000 QPS
 - Peak 1150 QPS requires about 2–3 servers (considering redundancy)
 - Total of about 15–20 servers needed (including all services)

High-level Design



Database



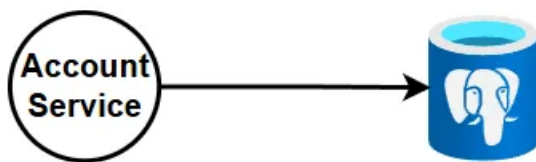
Users Table

```

1 CREATE TABLE users (
2     user_id VARCHAR(36) PRIMARY KEY,
3     email VARCHAR(255) NOT NULL UNIQUE,
4     password_hash VARCHAR(255) NOT NULL,
5     profile_data JSONB NOT NULL,
6     kyc_status ENUM('verified', 'pending', 'rejected') NOT NULL DEFAULT 'pending',
7     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
8 );

```

- `user_id` : Unique user identifier (UUID format), primary key
- `email` : User's email address, unique index
- `password_hash` : Encrypted password hash
- `profile_data` : User profile information including name, stored in JSON format
- `kyc_status` : Know Your Customer verification status (verified, pending, rejected)
- `created_at` : User creation timestamp



Accounts Table

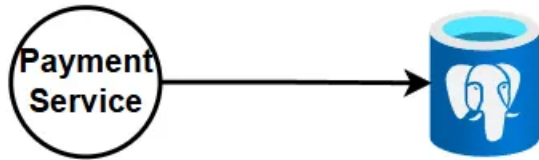
```

1 CREATE TABLE accounts (
2     account_id VARCHAR(10) PRIMARY KEY,
3     user_id VARCHAR(36) NOT NULL,
4     currency_code CHAR(3) NOT NULL,
5     balance DECIMAL(20,2) NOT NULL DEFAULT 0.00,
6     locked_balance DECIMAL(20,2) NOT NULL DEFAULT 0.00,
7     updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
8     FOREIGN KEY (user_id) REFERENCES users(user_id)
9 );

```

- `account_id` : Unique account identifier, primary key
- `user_id` : Associated user ID, foreign key
- `currency_code` : Currency code (e.g., USD, EUR, GBP, JPY)
- `balance` : Available balance

- `locked_balance` : Locked funds (being processed or frozen)
- `updated_at` : Last update timestamp



Payments Table

```

1  CREATE TABLE payments (
2      payment_request_id VARCHAR(10) PRIMARY KEY,
3      user_id VARCHAR(36) NOT NULL,
4      merchant_id VARCHAR(36) NOT NULL,
5      amount DECIMAL(20,2) NOT NULL,
6      currency_code CHAR(3) NOT NULL,
7      payment_method VARCHAR(50) NOT NULL,
8      request_time TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
9      status ENUM('pending', 'success', 'failed') NOT NULL DEFAULT 'pending'
10 ,
11     FOREIGN KEY (user_id) REFERENCES users(user_id)
12 );
  
```

- `payment_request_id` : Unique payment request identifier, primary key
- `user_id` : User initiating the payment, foreign key
- `merchant_id` : Merchant ID receiving the payment
- `amount` : Payment amount
- `currency_code` : Payment currency code
- `payment_method` : Payment method used (e.g., PayPal, Credit Card, Debit Card)
- `request_time` : Request initiation timestamp
- `status` : Payment status (pending, success, failed)

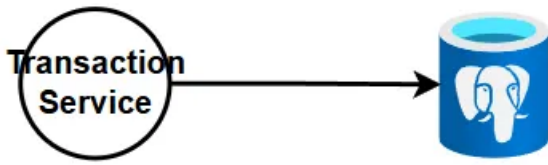
Currency Exchange Rates Table

```

1 CREATE TABLE currency_exchange_rates (
2     exchange_rate_id VARCHAR(10) PRIMARY KEY,
3     source_currency CHAR(3) NOT NULL,
4     target_currency CHAR(3) NOT NULL,
5     rate DECIMAL(10,6) NOT NULL,
6     last_updated TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
7 );

```

- `exchange_rate_id` : Unique exchange rate record identifier, primary key
- `source_currency` : Source currency code
- `target_currency` : Target currency code
- `rate` : Exchange rate value
- `last_updated` : Last update timestamp



Transaction Table

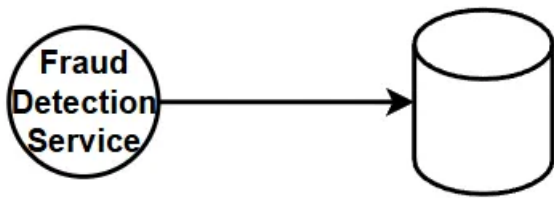
```

1 CREATE TABLE transactions (
2     transaction_id VARCHAR(10) PRIMARY KEY,
3     account_id VARCHAR(10) NOT NULL,
4     amount DECIMAL(20,2) NOT NULL,
5     currency_code CHAR(3) NOT NULL,
6     transaction_type ENUM('purchase', 'refund', 'deposit', 'withdrawal') NOT NULL,
7     status ENUM('pending', 'completed', 'failed') NOT NULL,
8     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
9     FOREIGN KEY (account_id) REFERENCES accounts(account_id)
10 );

```

- `transaction_id` : Unique transaction identifier, primary key
- `account_id` : Associated account ID, foreign key
- `amount` : Transaction amount
- `currency_code` : Transaction currency code
- `transaction_type` : Type of transaction (purchase, refund, deposit, withdrawal)

- `status` : Transaction status (pending, completed, failed)
- `created_at` : Transaction creation timestamp

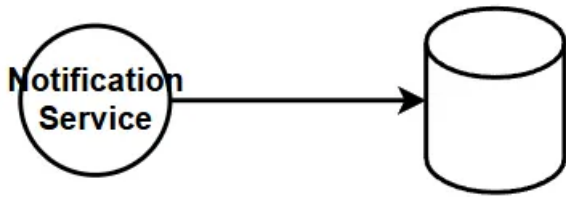


Fraud Detection Table

```

1  CREATE TABLE fraud_detection (
2      fraud_log_id VARCHAR(10) PRIMARY KEY,
3      transaction_id VARCHAR(10),
4      user_id VARCHAR(36) NOT NULL,
5      suspicious_activity VARCHAR(255) NOT NULL,
6      fraud_score INT NOT NULL,
7      action_taken VARCHAR(100) NOT NULL,
8      timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
9      FOREIGN KEY (transaction_id) REFERENCES transactions(transaction_id),
10     FOREIGN KEY (user_id) REFERENCES users(user_id)
11 );
  
```

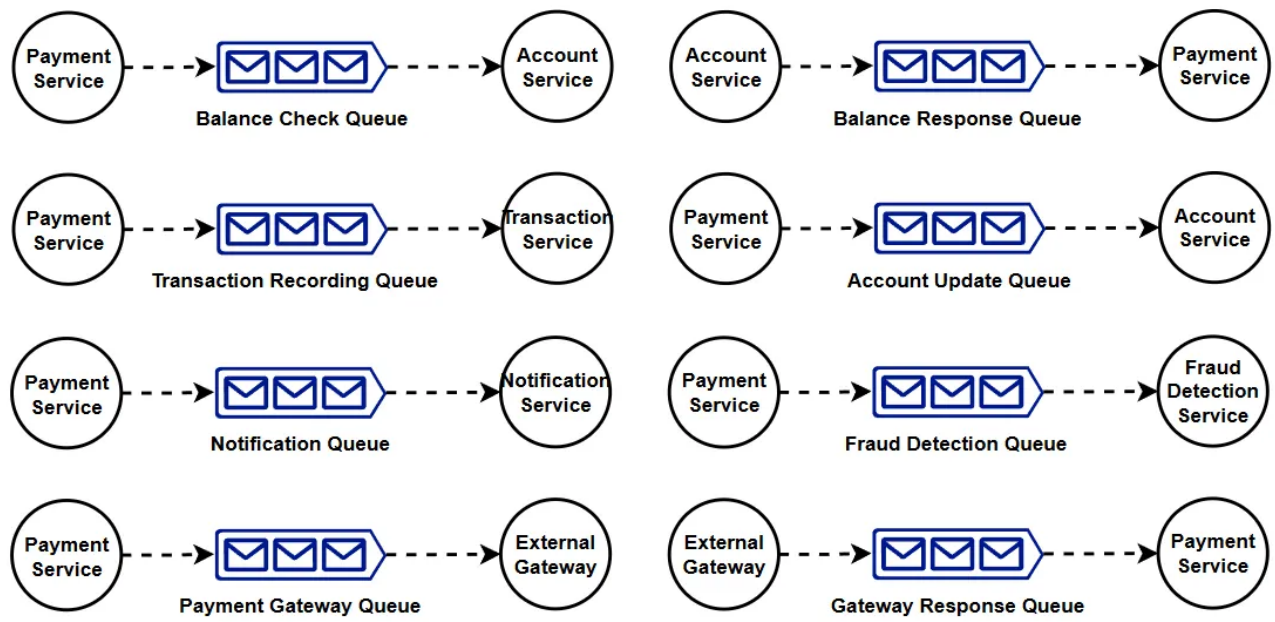
- `fraud_log_id` : Unique fraud log identifier, primary key
- `transaction_id` : Associated transaction ID (can be null if detection occurs before transaction), foreign key
- `user_id` : Associated user ID, foreign key
- `suspicious_activity` : Description of suspicious activity (e.g., Unusual Location, Multiple Failed Logins)
- `fraud_score` : Fraud risk score (0–100, higher score indicates higher risk)
- `action_taken` : Action taken (e.g., Payment Held, Flagged for Review, Payment Approved)
- `timestamp` : Record creation timestamp



Notification Table

```
1 CREATE TABLE notifications (  
2     notification_id VARCHAR(10) PRIMARY KEY,  
3     user_id VARCHAR(36) NOT NULL,  
4     notification_type ENUM('Email', 'SMS', 'Push', 'In-App') NOT NULL,  
5     message_content TEXT NOT NULL,  
6     delivery_status ENUM('Pending', 'Delivered', 'Failed') NOT NULL DEFAULT  
7     T 'Pending',  
8     timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
9     FOREIGN KEY (user_id) REFERENCES users(user_id)  
10 );
```

- `notification_id` : Unique notification identifier, primary key
- `user_id` : User receiving the notification, foreign key
- `notification_type` : Type of notification (Email, SMS, Push, In-App)
- `message_content` : Notification content
- `delivery_status` : Delivery status (Pending, Delivered, Failed)
- `timestamp` : Notification creation timestamp



Payment API Design

```
1  1. Payment Processing
2    POST /v1/payments
3    Headers:
4      Content-Type: application/json
5      Idempotency-Key: 12345-abcde-67890-fghij
6    Body:
7    {
8      "amount": 1000,
9      "currency": "USD",
10     "description": "Payment for order #12345"
11   }
12
13  2. Duplicate Payment Prevention (using idempotency)
14    POST /v1/payments
15    Headers:
16      Content-Type: application/json
17      Idempotency-Key: 12345-abcde-67890-fghij
18    Body:
19    {
20      "amount": 1000,
21      "currency": "USD",
22      "description": "Payment for order #12345"
23    }
24
25  3. Payment Query
26    GET /v1/payments/{payment_id}
27    Headers:
28      Content-Type: application/json
29
30
31  4. Refund
32    POST /v1/payments/{payment_id}/refunds
33    Headers:
34      Content-Type: application/json
35    Body:
36    {
37      "amount": 1000,
38      "reason": "Customer refund request"
39    }
```

